Studio Nx

Backend(FastAPI/Python) 58 — to_item() · list_projects()

2026-10-08

Backend(FastAPI/Python) 58 — `to_item()` · `list_projects()` (내 프로젝트 목록)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API — 목록 · 검색 · 정렬 · 페이지 · PRD 01 FR-123 · FR-130
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	57 `get_membership` · `require_role` · 59 `create_project`

1. 하는 일

- **`to_item`**: DB 의 `Project` 한 행을 화면이 쓰는 모양 `ProjectItem` 으로 바꾼다. 프로젝트 칸에 더해 세 칸을 계산해 넣는다.
- **`list_projects`**: 내가 멤버인 프로젝트를 검색 · 정렬 · 페이지로 잘라 `{items, total, page, size}` 로 돌려준다.

계산하는 칸	뜻	화면 (myProject.html)
`my_role`	이 프로젝트에서 내 역할 (OWNER · EDITOR · VIEWER)	카드의 역할 표시, 삭제 버튼은 OWNER 만
`member_count`	멤버 수	카드의 멤버 수
`opened`	**내가** 시퀀서에서 연 적이 있는지 (`project_sessions` 행이 있는지)	한 번도 안 열었으면 Not opened 표시 (FR-130)

요청 예	결과
`GET /projects`	최근 수정순, 1쪽 10개
`?sort=title`	이름순 (대소문자 무시: `alpha` · `Beta` · `Gamma`)
`?sort=created`	만든 순 (최근 것 먼저)
`?query=alp`	제목에 `alp` 가 들어간 것 (대소문자 무시)
`?query=%`	제목에 **글자** `%` 가 들어간 것만 (모든 프로젝트가 아님)
`?page=3` (2쪽까지만 있음)	`items` 빈 목록, `total` 은 그대로

2. 언제 쓰나

호출하는 곳	쓰는 것
`GET /api/v1/projects` 라우터 (62)	`list_projects`
`GET` · `POST` · `PATCH /projects/...` 응답 (62 · 59 · 60)	`to_item(db, project, member)` — 세 칸을 직접 DB 에 물어봄
`list_projects` 안	`to_item(..., member_count, opened)` — 미리 모아 계산한 값을 넘김
화면	myProject.html `loadProjects` · `renderProjects` (P1-14)

`project_sessions` 는 시퀀서를 열 때마다 생기는 작업 기록 (기록은 Phase 2). 지금은 행이 없으니 모든 프로젝트가 `opened=false` 로 나온다.

3. 코드

import (파일 맨 위)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, User
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`from sqlalchemy import func, select` 줄과 `ProjectSession` 을 이번에 추가했다. 두 함수는 `func` · `select` · `Session` · `Project` · `ProjectMember` · `ProjectSession` · `User` · `ProjectItem` · `ProjectListResponse` · `ProjectSort` 를 쓴다.

함수 (도움 함수 `_escape_like` · 정렬 표 `SORT_ORDER` 포함)

```
def to_item(
    db: Session,
    project: Project,
    member: ProjectMember,
    member_count: int | None = None,
    opened: bool | None = None,
) -> ProjectItem:
    if member_count is None:
        member_count = db.scalar(
            select(func.count())
            .select_from(ProjectMember)
            .where(ProjectMember.project_id == project.project_id)
        )
    if opened is None:
        session_id = db.scalar(
            select(ProjectSession.session_id)
            .where(
                ProjectSession.project_id == project.project_id,
                ProjectSession.user_id == member.user_id,
            )
            .limit(1)
        )
        opened = session_id is not None
    return ProjectItem(
        project_id=project.project_id,
        title=project.title,
        description=project.description,
        bpm=project.bpm,
```

```
        time_sig_num=project.time_sig_num,
        time_sig_den=project.time_sig_den,
        music_key=project.music_key,
        created_at=project.created_at,
        updated_at=project.updated_at,
        my_role=member.role,
        member_count=member_count,
        opened=opened,
    )


def _escape_like(text: str) -> str:
    return text.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")


SORT_ORDER = {
    ProjectSort.UPDATED: Project.updated_at.desc(),
    ProjectSort.TITLE: Project.title.asc(),
    ProjectSort.CREATED: Project.created_at.desc(),
}


def list_projects(
    db: Session,
    user: User,
    query: str,
    sort: ProjectSort,
    page: int,
    size: int,
) -> ProjectListResponse:
    conditions = [ProjectMember.user_id == user.user_id, Project.deleted_at.is_(None)]
    if query:
        conditions.append(Project.title.ilike(f"%{_escape_like(query)}%", escape="\\"))

    total = db.scalar(
        select(func.count())
        .select_from(Project)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(*conditions)
    )
    rows = db.execute(
        select(Project, ProjectMember)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(*conditions)
        .order_by(SORT_ORDER[sort], Project.project_id.desc())
        .offset((page - 1) * size)
        .limit(size)
    ).all()

    # 이 페이지 프로젝트들의 멤버 수 · 내가 연 적 있는지를 쿼리 두 번으로 모아 계산
    ids = [project.project_id for project, _ in rows]
    counts = {}
    opened_ids = set()
    if ids:
        counts = dict(
            db.execute(
                select(ProjectMember.project_id, func.count())
```

```
                .where(ProjectMember.project_id.in_(ids))
                .group_by(ProjectMember.project_id)
            ).all()
        )
        opened_ids = set(
            db.scalars(
                select(ProjectSession.project_id).where(
                    ProjectSession.project_id.in_(ids), ProjectSession.user_id == user.user_id
                )
            )
        )

    items = [
        to_item(db, project, member, counts[project.project_id], project.project_id in opened_ids)
        for project, member in rows
    ]
    return ProjectListResponse(items=items, total=total, page=page, size=size)
```

문법 풀이

import 문법	뜻
`from sqlalchemy import func, select`	`select(...)` = SELECT 문 만들기, `func.count()` = SQL `COUNT(*)`

식	뜻
`member_count: int | None = None`	안 넘기면 None → 함수 안에서 DB 에 물어봄
`db.scalar(select(...))`	결과의 **첫 줄 첫 칸** 하나만 (개수 · 번호 하나)
`.limit(1)` + `is not None`	행이 **하나라도 있는지**만 확인 (다 세지 않음)
`SORT_ORDER[sort]`	정렬 종류 → `ORDER BY` 식. `if/elif` 세 줄 대신 표로
`.where(*conditions)`	목록의 조건을 모두 AND 로 (`*` = 목록을 풀어 인자 여러 개로)
`Project.title.ilike(..., escape="\\")`	대소문자 무시 `LIKE`. 검색어 안의 `\` 뒤 글자는 특수 문자가 아니라 글자 그대로
`.offset((page - 1) * size).limit(size)`	3쪽 · 10개 → 앞 20개 건너뛰고 10개
`db.execute(select(Project, ProjectMember)...).all()`	한 줄에 `(project, member)` 두 객체
`.in_(ids)` · `.group_by(...)`	`WHERE project_id IN (...)` · 프로젝트별로 묶어 세기
`dict(rows)`	`[(5, 2), (8, 1)]` → `{5: 2, 8: 1}` (프로젝트 번호 → 멤버 수)
`project.project_id in opened_ids`	집합(set)에 있는지 → True / False

4. 흐름

1. 조건 만들기: 내가 멤버 · 삭제 안 됨 (+ 검색어가 있으면 제목 LIKE).
2. 같은 조건으로 **전체 개수**(`total`)를 센다.
3. 정렬 → 같은 값이면 `project_id` 큰 것 먼저 → 해당 쪽만 잘라 `(project, member)` 를 가져온다.
4. 이 쪽의 프로젝트 번호들로 **쿼리 두 번**: 멤버 수(묶어 세기), 내가 연 프로젝트 번호.
5. 줄마다 `to_item` 에 계산한 값을 넘겨 `ProjectItem` 을 만들고 응답.

목록 한 번에 쿼리는 **4번** (개수 · 목록 · 멤버 수 · 연 적 있음) — 프로젝트가 10개든 50개든 같다.

5. 왜 이렇게

선택	이유
멤버 수 · opened 를 **모아서** 계산	줄마다 물으면 10개 목록에 쿼리 20번이 더 나간다 (N+1 문제). 템플릿 권장
`to_item` 은 값을 안 넘기면 직접 물어봄	상세 · 만들기 · 수정 응답은 한 개라 모을 필요가 없다 — 한 함수를 두 곳에서 같이 씀
정렬 뒤에 `project_id DESC`	수정 시각이 같은 프로젝트들의 순서가 쿼리마다 바뀌면 2쪽에 1쪽 것이 또 나오거나 빠질 수 있다
검색어의 `%` · `_` · `\` 를 escape	`LIKE` 에서 `%` = 아무 글자 여럿, `_` = 아무 글자 하나. 그대로 두면 `%` 검색이 모든 프로젝트를 돌려준다 (템플릿 권장)
남이 만들었지만 내가 멤버인 것도 포함	FR-123 "내가 참여한 프로젝트" — 역할은 `my_role` 로 구분 (Phase 6 초대 대비)
`opened` 는 **내** 세션만	다른 멤버가 열었어도 나는 아직 안 열었으면 Not opened (FR-130 은 사람마다)
없는 쪽은 빈 목록 (404 아님)	마지막 프로젝트를 지운 뒤 그 쪽에 머물러도 화면이 오류가 아니라 "없음"을 보여 줄 수 있다

6. 시험 (`tests/test_project_service.py` · 13개 추가, 모두 25개)

시험	확인한 것
`to_item` 계산	멤버 2명 → `member_count` 2, 남이 연 세션은 무시, 내 세션이 생기면 `opened` True
`to_item` 값 넘기기	넘긴 7 · True 를 그대로 씀, 프로젝트 칸 복사
정렬 3가지	updated · title(대소문자 무시) · created 순서
같은 값	수정 · 만든 시각이 같으면 나중에 만든 것 먼저
페이지	2개씩: 1쪽 2개 · 2쪽 1개 · 3쪽 빈 목록, `total` 은 늘 3
검색	`ALP` → `alpha` (대소문자 무시)
특수 문자	`%` → `100% Mix` 만, `_` → `my_song` 만
범위	남의 것 · 삭제된 것 제외, 멤버로 들어간 남의 프로젝트는 EDITOR · 멤버 2명으로 포함
opened 목록	내가 연 것만 True
빈 목록	프로젝트가 없으면 `items=[]`, `total=0`

전체 시험 463개 통과, `ruff` 검사 통과.